

รายงาน Lab 10 Spring WebFlux และ WebClient

รายวิชา: CP353002 Principles of Software Design

เรื่อง: ระบบจัดการสินค้าแบบ Non-blocking Reactive Programming

ชื่อ-นามสกุล: นายปวัฒน์ ปัทมมา

รหัสนักศึกษา: 673380048-6 Section: 1

ส่วนที่ 1 หลักการและการออกแบบ

1.1 ภาพรวมของระบบ

พัฒนา REST API สำหรับสร้าง อ่าน ลบ และค้นหาสินค้าตามหมวดหมู่ รวมถึงคำนวณราคาหลังส่วนลด โดยใช้ Spring WebFlux คีนค่า Mono และ Flux และใช้ WebClient เรียก API แบบ Reactive การทดสอบยืนยันการทำงานผ่าน HTTP และการต่อ operators ได้ตามโจทย์

1.2 Reactive Programming และ Blocking

Blocking I/O ทำให้ thread ที่เรียกต้องรอจนการรับส่งข้อมูลเสร็จ ส่วน Non-blocking I/O ไม่ยึด thread ไว้ระหว่างรอเครือข่าย เมื่อข้อมูลพร้อมจึงดำเนินงานต่อผ่านลำดับ operators ระบบนี้ใช้ Reactor Netty รับ HTTP และ WebClient ส่ง HTTP แบบ non-blocking

ประเด็น	Blocking	Reactive ใน Lab นี้
การรอ I/O	thread รอผลการเรียก	ดำเนินงานต่อเมื่อมี signal
รูปแบบผลลัพธ์	คีนค่าหลังทำงานเสร็จ	คีน Mono หรือ Flux
การจัดการข้อมูล	ลำดับคำสั่งทั่วไป	ต่อ map filter flatMap
สิ่งที่ต้องระวัง	ใช้ thread มากจนรอ	ห้ามใส่ blocking I/O ใน chain

การคีน Mono หรือ Flux เพียงอย่างเดียวไม่ได้ทำให้โค้ดภายในเป็น non-blocking โดยอัตโนมัติ ใน endpoint นี้ไม่มีการใช้ .block(), Thread.sleep(), JDBC หรือ JPA และใช้ข้อมูลในหน่วยความจำตาม Template

1.3 Mono และ Flux

ชนิด	จำนวนข้อมูล	การใช้ในระบบ
Mono<Product>	0 หรือ 1 รายการ	ค้น ID และบันทึกสินค้า
Flux<Product>	0 ถึงหลายรายการ	ดูทั้งหมดและค้น category
Mono<Double>	0 หรือ 1 ค่า	ราคาหลังส่วนลด
Mono<Void>	ไม่มีค่าข้อมูล	แจ้งว่าการลบเสร็จหรือผิดพลาด

Publisher ส่ง onNext เมื่อมีข้อมูล ตามด้วย onComplete เมื่อจบตามปกติ หรือ onError

เมื่อเกิดข้อผิดพลาด Mono.empty() จบโดยไม่ส่งข้อมูล จึงต่างจาก Mono.error() ที่จบด้วย error

1.4 Operators ที่ใช้

Operator	หน้าที่และตัวอย่างในระบบ
map	แปลง Product เป็นราคาหรือชื่อ โดยไม่คืน Publisher ซ้อน
flatMap	นำ Product ไปเรียก repository.save() หรือเรียก HTTP ต่อ
filter	เลือกสินค้าตาม category หรือเลือกเฉพาะที่มี stock
switchIfEmpty	เปลี่ยน Mono ว่างเป็น error เมื่อค้น ID ไม่พบ
defaultIfEmpty	ให้ข้อความสำรองเมื่อหมดสต็อกไม่มีสินค้า
doOnNext	แสดงราคาที่ได้รับได้ โดยไม่เปลี่ยนค่าที่ส่งต่อ
subscribe	เริ่มรับผลในตัวอย่าง client พร้อม callback จัดการ error

Repository ใช้ Mono.defer, Flux.defer, Mono.fromSupplier และ Mono.fromRunnable

เพื่อเลื่อนการอ่านหรือเปลี่ยนข้อมูลไปทำเมื่อมีการ subscribe ส่วน Controller คั้น Publisher ให้ Spring
WebFlux subscribe เพื่อสร้าง HTTP response จึงไม่ต้องเรียก subscribe เองใน Controller

flatMap ใช้ต่อ Publisher อีกตัว แต่ไม่ได้หมายความว่าสร้าง thread ใหม่เสมอไป ส่วน defaultIfEmpty
ทำงานเฉพาะ stream ว่าง ไม่ใช่ตัวดัก HTTP error

1.5 โครงสร้างและข้อมูลสินค้า

Controller รับ HTTP request แล้วเรียก Service ซึ่งจัดการกฎของงาน เช่น สร้าง UUID และคำนวณราคา จากนั้น Repository อ่านหรือแก้ไขข้อมูลใน ConcurrentHashMap ส่วน ProductWebClient เป็นผู้เรียก HTTP API ของระบบ โดยแยกหน้าที่จาก Controller

ไฟล์หรือชั้น	หน้าที่
Product	ข้อมูล id name category brand stock price discountType
ProductRepository	เก็บข้อมูลและคืน Mono หรือ Flux
ProductService	จัดการ ID กรณีไม่พบ และราคาหลังลด
ProductController	ผูก 6 endpoints กับ Service
ProductWebClient	เรียก API ผ่าน HTTP และคืน Publisher
AppConfig	ประกาศ ProductRepository เป็น Spring Bean

โปรเจกต์ใช้ Spring Boot 3.3.0 และ Java target 17 พร้อม spring-boot-starter-webflux โดยไม่เพิ่ม spring-boot-starter-web ชื่อ artifactId คือ lab10-673380048-6-sec1 และใช้ port 8080

1.6 ข้อมูลตั้งต้นและส่วนลด

ID	สินค้า	ราคา	ส่วนลด	ราคาหลังลด
1	iPhone 15 Pro	39900	MEMBER 10%	35910
2	MacBook Air M3	49900	NONE	49900
3	Samsung Galaxy S24	29900	SEASONAL 20%	23920

ชื่อสินค้า ID 1 มีข้อความ “ปวัฒน์ ปัดทุมมา 673380048-6 SEC 1” เพื่อให้เห็นชื่อและรหัสใน response ตาม README ข้อมูลเก็บในหน่วยความจำ ไม่ใช่ PostgreSQL และจะกลับเป็นข้อมูลตั้งต้นเมื่อเริ่มโปรแกรมใหม่

Product.getDiscountedPrice() คืน $\text{price} \times 0.90$ สำหรับ MEMBER, $\text{price} \times 0.80$ สำหรับ SEASONAL และราคาเดิมสำหรับค่าอื่น หาก price เป็น null จะคืน 0.0 ตาม helper ที่อาจารย์ให้มา

ส่วนที่ 2 โค้ดและคำอธิบาย

2.1 ProductRepository

แสดงส่วนเมธอดที่เติมตาม TODO โดยละ imports และข้อมูล seed เพื่อให้อ่านง่าย

คอมเมนต์เดิมของอาจารย์ยังคงอยู่ครบใน source code

```
// เก็บข้อมูลในหน่วยความจำตาม Template
private final Map<String, Product> store = new ConcurrentHashMap<>();

public Mono<Product> findById(String id) {
    // อ่านเมื่อ subscribe และคืน Mono ว่างหากไม่พบ
    return Mono.defer() -> Mono.justOrEmpty(store.get(id));
}

public Flux<Product> findAll() {
    return Flux.defer() -> Flux.fromIterable(store.values());
}

public Mono<Product> save(Product product) {
    // บันทึกเมื่อ subscribe แล้วส่งสินค้าที่บันทึกต่อ
    return Mono.fromSupplier() -> {
        store.put(product.getId(), product);
        return product;
    };
}

public Mono<Void> deleteById(String id) {
    return Mono.fromRunnable() -> store.remove(id);
}

public Flux<Product> findByCategory(String category) {
    // กรองโดยไม่แยกตัวพิมพ์ใหญ่และเล็ก
    return findAll()
        .filter(product -> product.getCategory() != null
            && product.getCategory().equalsIgnoreCase(category));
}
```

findById ไม่คืน null ให้ผู้เรียก แต่คืน Mono.empty() เมื่อไม่มีข้อมูล save และ deleteById เปลี่ยน store เมื่อมี subscriber ส่วน findByCategory ใช้ filter กับ Flux ที่ได้จาก findAll และตรวจสอบ category ไม่เป็น null ก่อนเปรียบเทียบ

ConcurrentHashMap ไม่รับประกันลำดับรายการ ดังนั้นผล GET ทั้งหมดอาจเรียงต่างกันได้

การทดสอบจึงไม่ควรยึดลำดับสินค้าโดยไม่จำเป็น

2.2 ProductService

```
@Service
public class ProductService {
    private final ProductRepository repository;
    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }
    public Mono<Product> getByld(String id) {
        // เปลี่ยนกรณีไม่พบข้อมูลให้เป็น error ตามโจทย์
        return repository.findByld(id)
            .switchIfEmpty(Mono.error() ->
                new RuntimeException("Product not found: " + id));
    }
    public Flux<Product> getAll() {
        return repository.findAll();
    }
    public Mono<Product> save(Product product) {
        return Mono.fromSupplier() -> {
            // คง ID ที่ส่งมา หรือสร้าง UUID เมื่อ ID เป็น null
            if (product.getId() == null) {
                product.setId(java.util.UUID.randomUUID().toString());
            }
            return product;
        }).flatMap(repository::save);
    }
    public Mono<Void> delete(String id) {
        return repository.deleteByld(id);
    }
    public Flux<Product> getByCategory(String category) {
        return repository.findByCategory(category);
    }
    public Mono<Double> getDiscountedPrice(String id) {
        return repository.findByld(id).map(Product::getDiscountedPrice);
    }
}
```

ใช้ constructor injection รับ Repository และเก็บใน final field การบันทึกใช้ flatMap เพราะ repository.save คืน Mono<Product> ส่วนการคำนวณราคาใช้ map เพราะ getDiscountedPrice ของ Product คืน Double ธรรมดา

เมื่อค้น ID ไม่พบ Service ส่ง RuntimeException ตาม Template ปัจจุบันไม่มีตัวแปลงเป็น 404 จึงตอบ HTTP 500 การค้น category ที่ไม่มีข้อมูลกลับคืน Flux ว่างตามปกติ

2.3 ProductController

```
@RestController
@RequestMapping("/products")
public class ProductController {
    private final ProductService service;
    public ProductController(ProductService service) {
        this.service = service;
    }
    // คีนสินค้า 1 รายการ
    @GetMapping("/{id}")
    public Mono<Product> getById(@PathVariable String id) {
        return service.getById(id);
    }
    // คีนสินค้าหลายรายการ
    @GetMapping
    public Flux<Product> getAll() {
        return service.getAll();
    }
    @PostMapping
    public Mono<Product> save(@RequestBody Product product) {
        return service.save(product);
    }
    @DeleteMapping("/{id}")
    public Mono<Void> delete(@PathVariable String id) {
        return service.delete(id);
    }
    @GetMapping("/category/{category}")
    public Flux<Product> getByCategory(@PathVariable String category) {
        return service.getByCategory(category);
    }
    @GetMapping("/{id}/price")
    public Mono<Double> getDiscountedPrice(@PathVariable String id) {
        return service.getDiscountedPrice(id);
    }
}
```

Controller คีน Mono หรือ Flux ให้ framework จัดการ response โดยตรง ไม่มีการ block รอค่า และไม่ subscribe เอง เมธอด POST รับ JSON เป็น Product ก่อนส่งให้ Service ส่วน DELETE คีน Mono<Void> และในโค้ดนี้ตอบ 200 พร้อม body ว่าง เพราะไม่ได้กำหนดสถานะ 204 เพิ่ม

2.4 ProductWebClient

แต่ละเมธอดคืน Publisher ให้ผู้เรียกนำไปต่อ operators ได้ ส่วนการ subscribe อยู่ที่ผู้ใช้ client หรือ framework ไม่ได้อยู่ในเมธอดเรียก HTTP เหล่านี้

```
private final WebClient client = WebClient.create("http://localhost:8080");

public Mono<Product> getProductById(String id) {
    return client.get().uri("/products/{id}", id)
        .retrieve().bodyToMono(Product.class);
}

public Flux<Product> getAllProducts() {
    return client.get().uri("/products")
        .retrieve().bodyToFlux(Product.class);
}

public Mono<Product> createProduct(Product product) {
    // ส่ง Product เป็น request body
    return client.post().uri("/products").bodyValue(product)
        .retrieve().bodyToMono(Product.class);
}

public Mono<Void> deleteProduct(String id) {
    return client.delete().uri("/products/{id}", id)
        .retrieve().bodyToMono(Void.class);
}

public Flux<Product> getByCategory(String category) {
    return client.get().uri("/products/category/{category}", category)
        .retrieve().bodyToFlux(Product.class);
}

public Mono<Double> getDiscountedPrice(String id) {
    return client.get().uri("/products/{id}/price", id)
        .retrieve().bodyToMono(Double.class)
        .doOnNext(price -> System.out.println("Price: " + price));
}
```

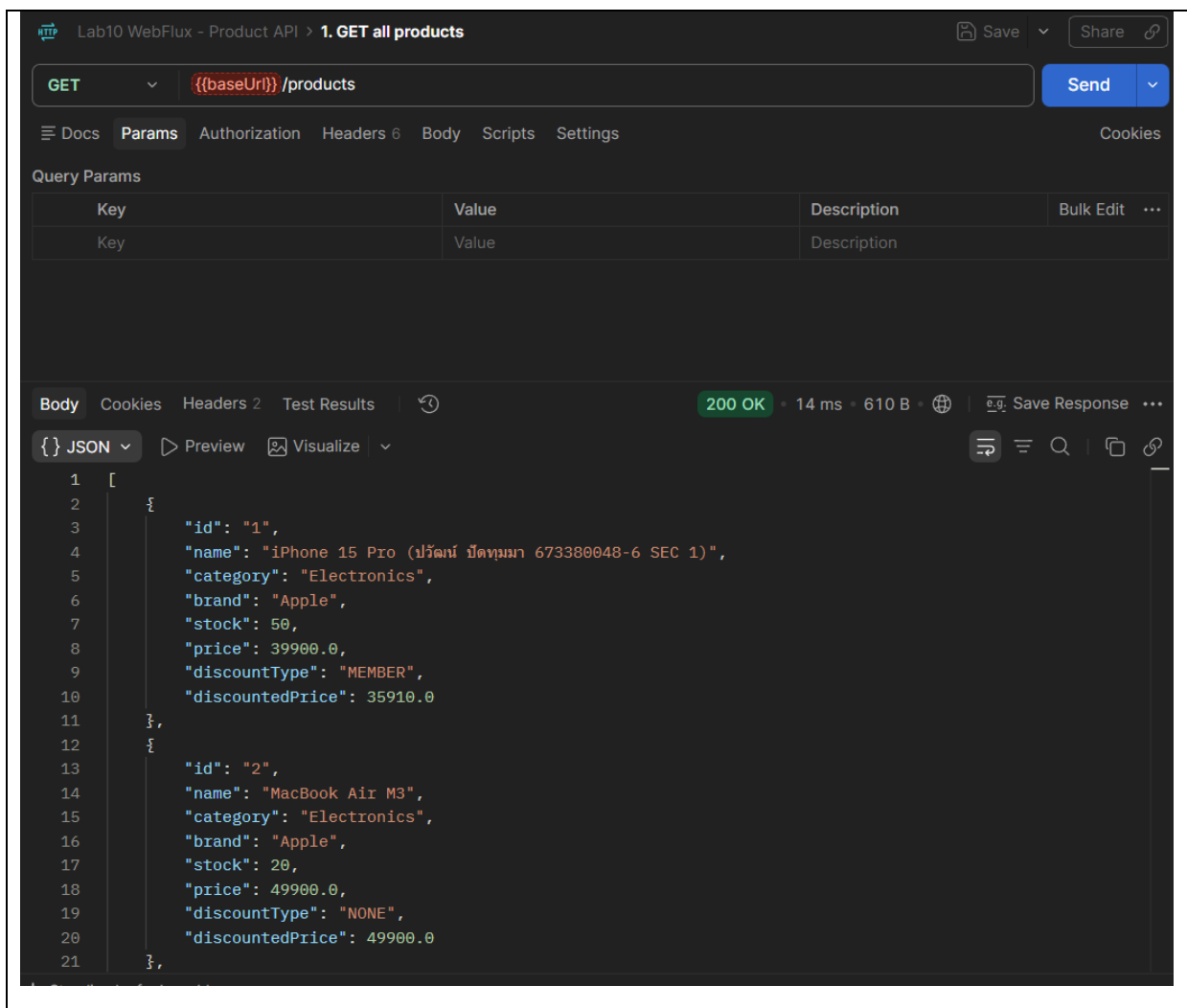
ส่วนของ chain	คำอธิบาย
get post delete	กำหนด HTTP method
uri	กำหนด path และแทนค่าตัวแปรใน URL
bodyValue	กำหนดข้อมูล JSON สำหรับ POST
retrieve	กำหนดการอ่าน response และจัดการ HTTP error
bodyToMono / bodyToFlux	ถอด response เป็นข้อมูลหนึ่งค่าหรือหลายค่า
doOnNext	ทำ side effect เพื่อแสดงราคาที่รับได้

ส่วนที่ 3 ผลการทดสอบ API

3.1 ดูลิ้นคำทั้งหมด

เริ่มโปรแกรมด้วย `mvn spring-boot:run` ในโฟลเดอร์ `LAB10_WebClient` แล้วใช้ Postman ส่ง GET `http://localhost:8080/products`

ลำดับภาพ 1–6 ให้ทดสอบต่อเนื่องในรอบเดียว โดยสร้างสินค้าทดสอบก่อนลบ และไม่ลบสินค้าตั้งต้น ID 1–3 ผลในคำอธิบายตรวจจาก HTTP และ tests แล้ว ช่องภาพเว้นไว้สำหรับหลักฐานจาก Postman ที่นำมาใส่ภายหลัง



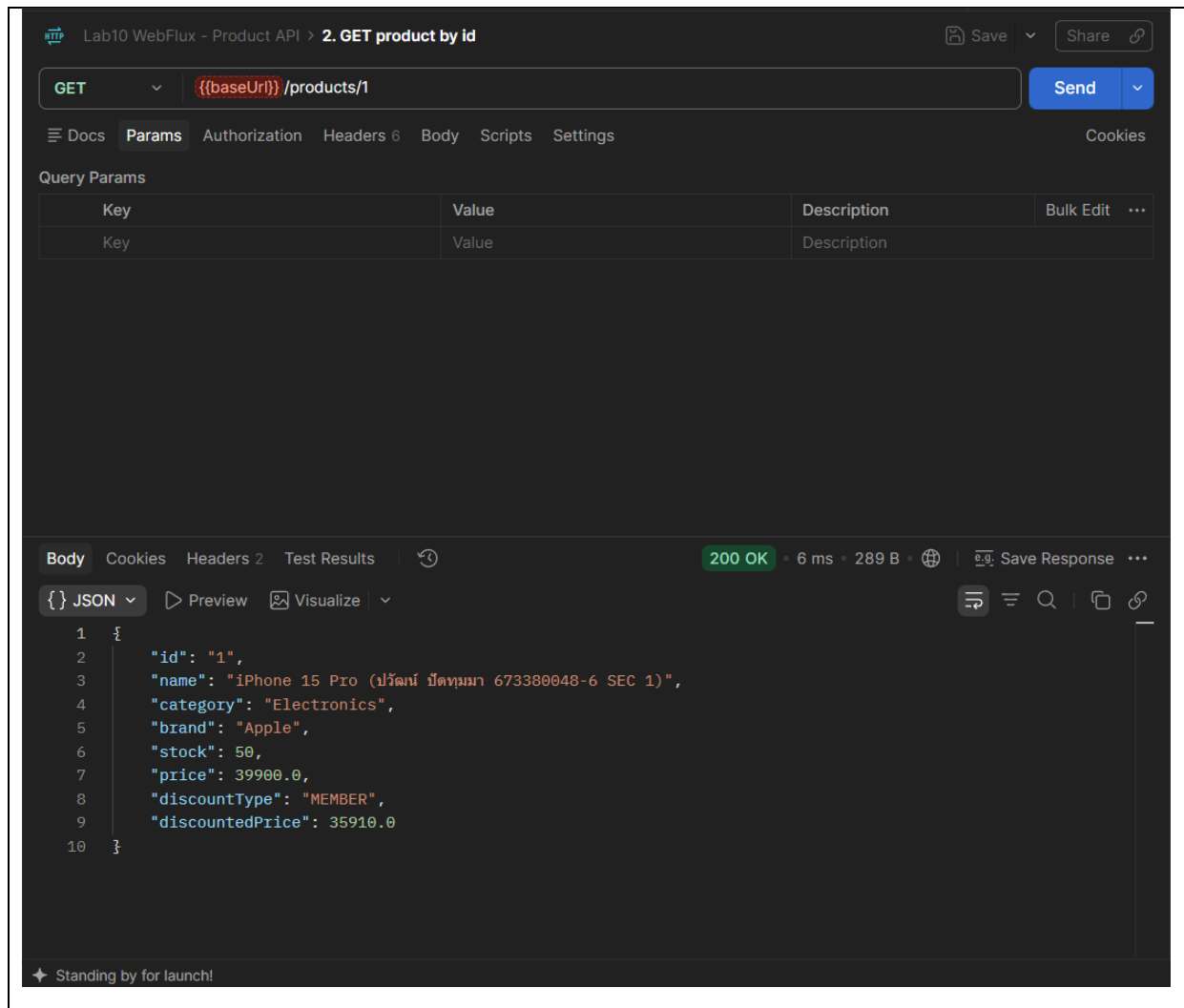
ภาพที่ 1 GET /products แสดงสินค้าตั้งต้น 3 รายการและ Status 200

ผลตรวจสอบ: API ส่ง JSON array ของสินค้าตั้งต้น 3 รายการ ได้แก่ iPhone 15 Pro, MacBook Air M3 และ Samsung Galaxy S24 โดยซื้อสินค้า ID 1 มีชื่อและรหัสนักศึกษา ผลลัพธ์มาจาก Flux<Product> และลำดับรายการอาจเปลี่ยนได้

จุดที่ต้องเห็นในภาพ: Method, URL, Status 200 OK และ response ที่แสดงข้อมูลสินค้า รวมถึงชื่อ-รหัสในสินค้า ID 1

3.2 ดูลิ้นค้าตาม ID

ส่ง GET `http://localhost:8080/products/1` โดยไม่ต้องใส่ Body



ภาพที่ 2 GET `/products/1` แสดงชื่อ รหัสนักศึกษา และข้อมูลสินค้า ID 1

ผลตรวจสอบ: API ส่งสินค้า ID 1 เป็น JSON object จาก `Mono<Product>` มีชื่อ iPhone 15 Pro (ปวัฒน์ บัดหุมมา 673380048-6 SEC 1), category เป็น Electronics, stock เท่ากับ 50 และ price เท่ากับ 39900.0

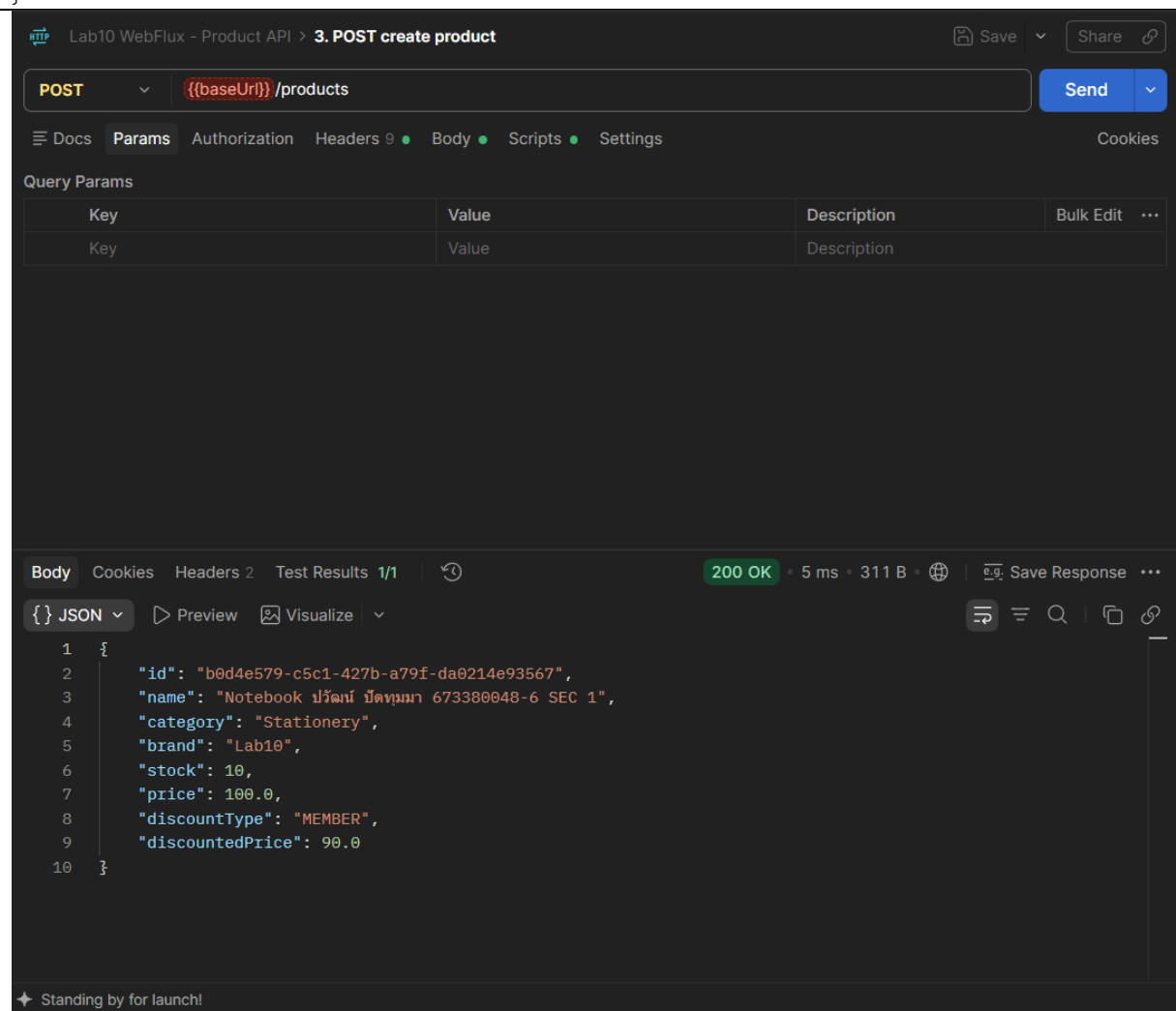
Product มี getter `getDiscountedPrice` จึงอาจเห็นฟิลด์ `discountedPrice` เท่ากับ 35910.0 ใน JSON ด้วย การเรียก endpoint นี้ไม่ได้เปลี่ยนราคาเดิมหรือ stock

จุดที่ต้องเห็นในภาพ: URL `/products/1`, Status 200 OK และชื่อ-รหัสนักศึกษาใน response อย่างชัดเจน

3.3 สร้างสินค้า

ส่ง POST `http://localhost:8080/products` เลือก Body แบบ raw และ JSON โดยใช้ข้อมูลดังนี้ ไม่ส่ง id เพื่อให้ Service สร้าง UUID

```
{
  "name": "Notebook ปวดันน์ ปัดทุมมา 673380048-6 SEC 1",
  "category": "Stationery",
  "brand": "Lab10",
  "stock": 10,
  "price": 100.0,
  "discountType": "MEMBER"
}
```



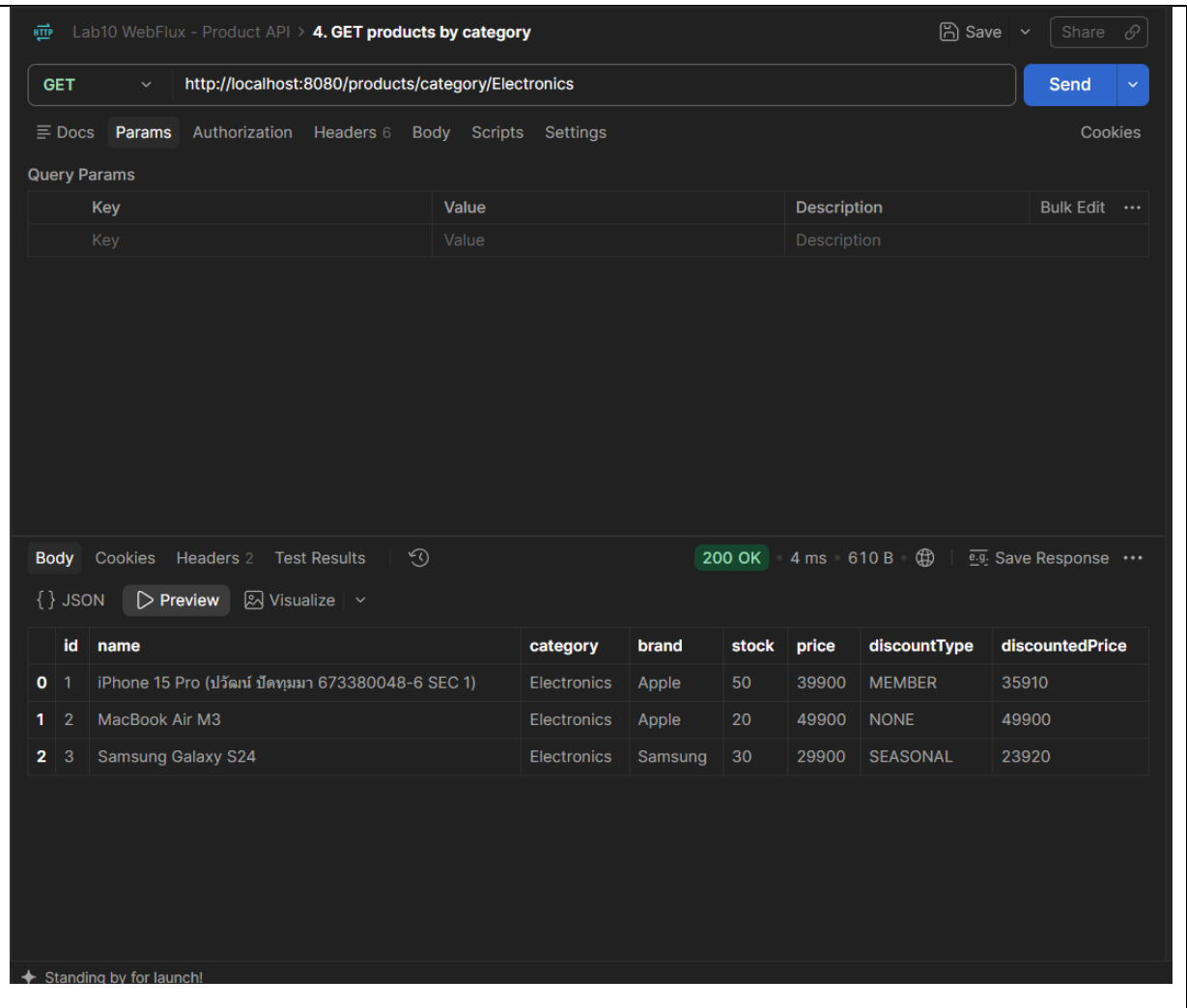
ภาพที่ 3 POST /products แสดง JSON ที่ส่งและสินค้าที่สร้างพร้อม UUID

พฤติกรรมที่ตรวจสอบแล้ว: เมื่อไม่ส่ง id ระบบสร้าง UUID บันทึกสินค้า และตอบ 200 OK พร้อม Product ที่สร้าง ในคำขอตัวอย่างนี้ราคาหลังลดควรเป็น 90.0 ให้เก็บ id จาก response ไว้ใช้ลบในหัวข้อ 3.6; ค่า UUID ในแต่ละรอบไม่เหมือนกัน

ผล automated test ตรวจสอบการสร้างและอ่านกลับผ่าน WebClient สำเร็จ ส่วนภาพนี้แสดงผลจริงของ JSON ตัวอย่างข้างต้น

3.4 คั่นสินค้าตามหมวดหมู่

ส่ง GET `http://localhost:8080/products/category/Electronics`



Lab10 WebFlux - Product API > 4. GET products by category

GET `http://localhost:8080/products/category/Electronics` Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 2 Test Results 200 OK • 4 ms • 610 B • Save Response

{ } JSON Preview Visualize

	id	name	category	brand	stock	price	discountType	discountedPrice
0	1	iPhone 15 Pro (ปริ้นท์ บัดทุมมา 673380048-6 SEC 1)	Electronics	Apple	50	39900	MEMBER	35910
1	2	MacBook Air M3	Electronics	Apple	20	49900	NONE	49900
2	3	Samsung Galaxy S24	Electronics	Samsung	30	29900	SEASONAL	23920

Standing by for launch!

ภาพที่ 4 GET ตาม category Electronics แสดงสินค้าในหมวดหมู่ 3 รายการ

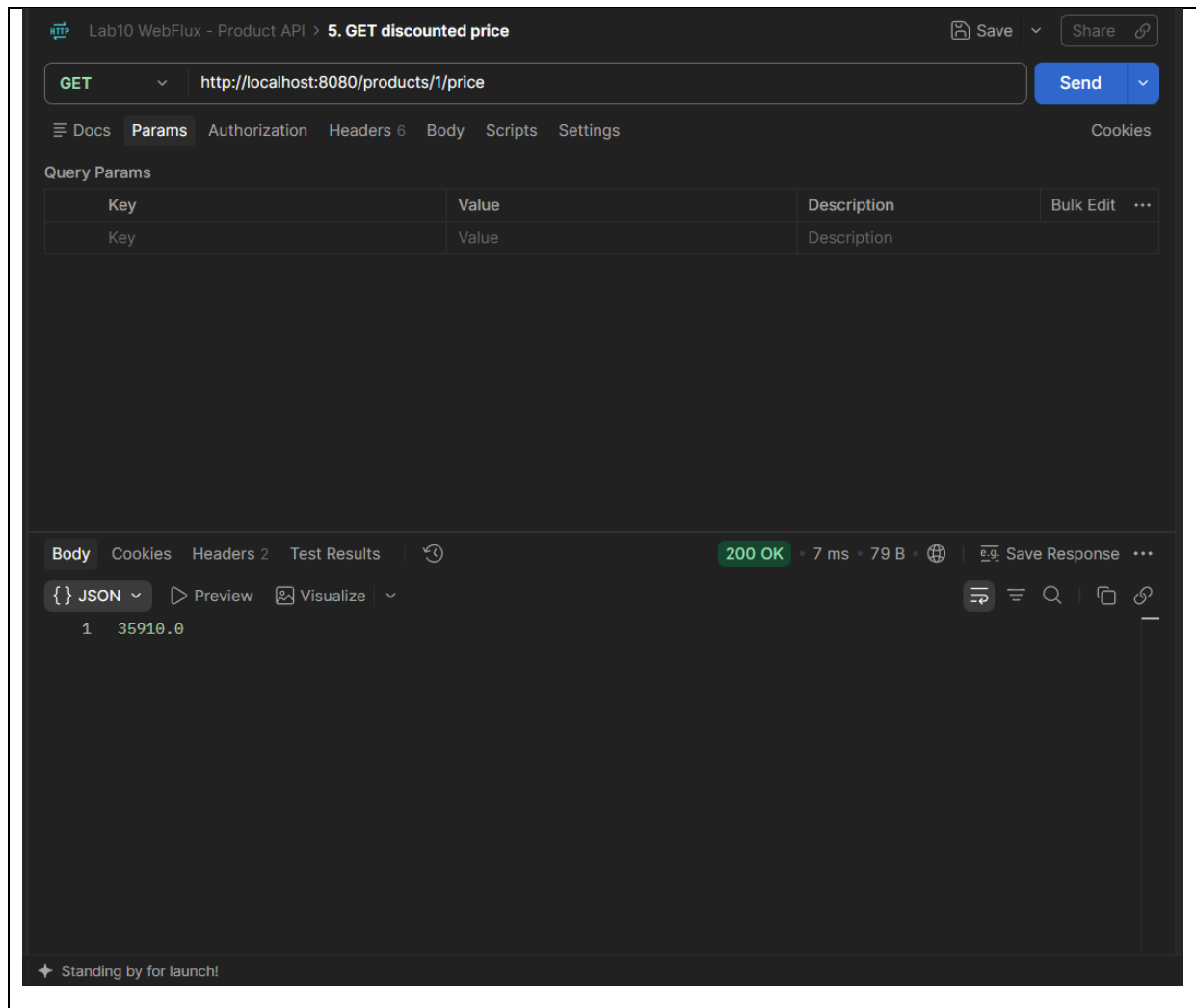
ผลตรวจสอบ: คั่นสินค้าตั้งต้น 3 รายการในหมวด Electronics เท่านั้น สินค้า Notebook ที่สร้างเป็น Stationery จึงไม่รวมในผลนี้ Repository ใช้ filter และ equalsIgnoreCase จึงค้นด้วย electronics ตัวพิมพ์เล็กได้เช่นกัน

ถ้าค้นหมวดที่ไม่มี เช่น `/products/category/NoSuchCategory` จะได้ 200 OK และ array ว่าง ไม่ใช่ error ตัวอย่าง WebClient ใช้กรณีนี้สาริต `defaultIfEmpty`

จุดที่ต้องเห็นในภาพ: URL ที่ระบุ category, Status และฟิลด์ category ของสินค้าที่ได้กลับมา

3.5 ดูราคาหลังส่วนลด

ส่ง GET `http://localhost:8080/products/1/price`



ภาพที่ 5 GET `/products/1/price` แสดงราคาหลังส่วนลด 35910.0

ผลตรวจสอบ: ได้ 200 OK และตัวเลข 35910.0 จากราคาเดิม 39900.0 ลด MEMBER 10 เปอร์เซ็นต์ Service ใช้ map แปลง Product เป็น Double โดยเรียก `getDiscountedPrice` และไม่แก้ price เดิมในสินค้า

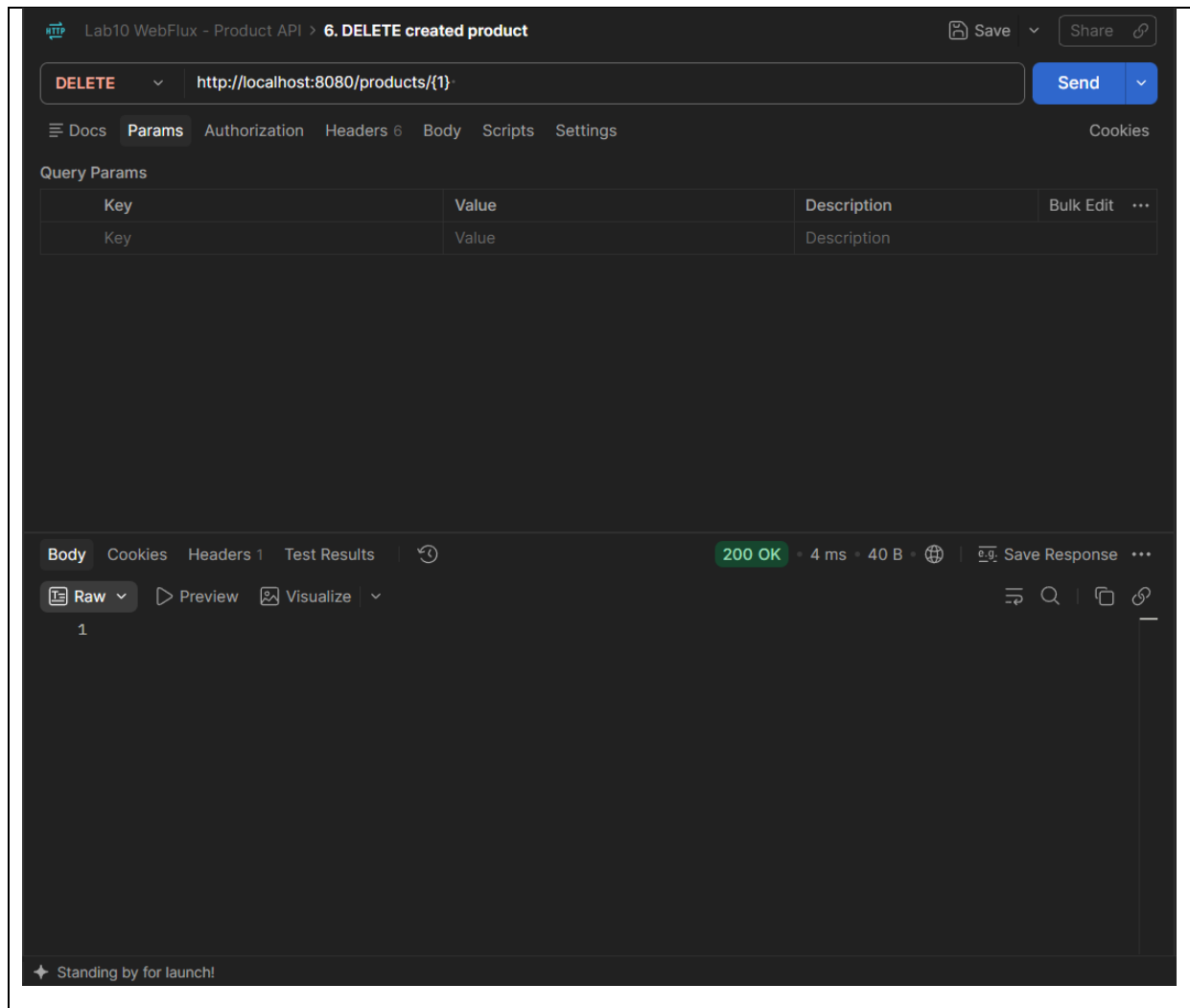
กรณีทดสอบเพิ่มเติม	ค่าที่ตรวจสอบได้
ID 1 MEMBER	$39900 \times 0.90 = 35910$
ID 2 NONE	49900 เท่าเดิม
ID 3 SEASONAL	$29900 \times 0.80 = 23920$

จุดที่ต้องเห็นในภาพ: URL ที่ส่งท้าย `/price`, Status 200 OK และค่าตัวเลขใน response

3.6 ลบสินค้า

ส่ง DELETE http://localhost:8080/products/{id} โดยแทน {id} ด้วย UUID ของ Notebook จากหัวข้อ

3.3 ไม่ต้องใส่ Body



ภาพที่ 6 DELETE สินค้าที่สร้าง แสดง URL พร้อม UUID และ Status 200

ผลตรวจสอบการลบ: Mono<Void> จบโดยไม่ส่งข้อมูล API ตอบ 200 OK และ body ว่างตาม Controller

ปัจจุบัน จากนั้นเรียก GET /products เพื่อตรวจว่าสินค้า UUID นั้นหายไป

หากทำตามลำดับนี้และไม่ได้สร้างเพิ่ม ควรเหลือสินค้าตั้งต้น 3 รายการ

หากเรียก GET ด้วย ID ที่ลบแล้ว จะได้ HTTP 500 เนื่องจาก Service ส่ง RuntimeException ตาม

Template ผลนี้ต่างจากคั่น category ที่ไม่พบซึ่งส่ง array ว่าง

จุดที่ต้องเห็นในภาพ: Method DELETE, URL ที่ใช้ UUID ของสินค้าทดสอบ และสถานะตอบกลับ
หากต้องการเพิ่มหลักฐาน สามารถวางภาพ GET /products หลังลบต่อท้ายได้

ภาพที่ 7 Console แสดงข้อสินค้า ข้อความหมวดหมู่ว่าง และราคาหลังลด

defaultIfEmpty ไม่แปลง error เป็นค่าทดแทน หากเรียก ID ที่ไม่พบ WebClient จะได้รับ HTTP 500 และเข้า error callback; ตัวอย่างหมวดหมู่ว่างจึงใช้แสดง fallback ที่เกิดขึ้นจริง

4.2 ผลการทดสอบอัตโนมัติ

รัน mvn test ผลที่ตรวจสอบล่าสุดผ่าน 21 กรณี Failures 0 และ Errors 0 โดยใช้ StepVerifier ตรวจสอบค่า onNext, onComplete และ onError พร้อม integration test เปิด HTTP server บน port สุ่มแยกจากโปรแกรมที่ใช้แคปภาพ

ชุดทดสอบ	จำนวน	สิ่งที่ตรวจ
Lab10ApplicationTests	6	Context และ TODO ทดสอบ Repository ของอาจารย์
ProductRepositoryTest	4	ข้อมูลตั้งต้น การ subscribe บันทึก ลบ และ category
ProductServiceTest	6	UUID ID เดิม ส่วนลด และกรณีไม่พบ
ProductWebClientIntegrationTest	5	HTTP จริง ครบ 6 endpoints และ fallback
รวม	21	ผ่านทั้งหมด ไม่มี failure หรือ error

```
Windows PowerShell
2026-09-17T01:48:54.846+07:00 INFO 34732 --- [lab10-webflux] [main] c.example.lab10.Lab10ApplicationTests
: Starting Lab10ApplicationTests using Java 21.0.8 with PID 34732 (started by Pawat in K:\CP353002-69_1PrinciplesOfSoft
wareDesign\LAB10_WebClient)
2026-09-17T01:48:54.846+07:00 DEBUG 34732 --- [lab10-webflux] [main] c.example.lab10.Lab10ApplicationTests
: Running with Spring Boot v3.3.0, Spring v6.1.8
2026-09-17T01:48:54.846+07:00 INFO 34732 --- [lab10-webflux] [main] c.example.lab10.Lab10ApplicationTests
: No active profile set, falling back to 1 default profile: "default"
2026-09-17T01:48:54.986+07:00 INFO 34732 --- [lab10-webflux] [main] c.example.lab10.Lab10ApplicationTests
: Started Lab10ApplicationTests in 0.167 seconds (process running for 3.394)
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.195 s -- in com.example.lab10.Lab10ApplicationT
ests
[INFO] Running com.example.lab10.repository.ProductRepositoryTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.007 s -- in com.example.lab10.repository.Produc
tRepositoryTest
[INFO] Running com.example.lab10.service.ProductServiceTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.012 s -- in com.example.lab10.service.ProductSe
rviceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 21, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 7.348 s
[INFO] Finished at: 2569-09-17T01:48:57+07:00
[INFO]
PS K:\CP353002-69_1PrinciplesOfSoftwareDesign\LAB10_WebClient> |
```

ภาพที่ 8 ผล mvn test แสดง Tests run 21 และ BUILD SUCCESS

4.3 สรุปผล

ระบบทำงานครบ 6 endpoints และใช้ WebClient ต่อ operators ได้ การทดสอบผ่านทั้ง UUID, category, ส่วนลด และการสร้างแล้วลบสินค้า โค้ดหลักไม่มี .block() หรือ Thread.sleep() และเก็บคอมเมนต์อาจารย์ครบ